

Flutter Development Log

ES — Software Engineering

March 9, 2026

Contents

1	User Story Selection	2
2	Business Value Analysis	2
3	Dependencies and Assumptions	2
4	Scenario Design	2
5	UI Mockups	2
6	Acceptance Tests	3
7	PDF Deliverable	3
8	Critical Analysis	3

1 User Story Selection

The first step was reviewing my teammates' user stories for the MatchWatch app and selecting two to work with. After going through them, I picked the **Friends** feature and the **Chat Functionality** feature, as both are closely related and together they represent a meaningful slice of the app's social functionality.

2 Business Value Analysis

Before writing anything, I had to understand what *business value* meant in the context of user stories — specifically, why a feature matters to the product beyond its surface functionality.

I researched the concept and thought about how it applied to each chosen story:

- **Friends:** The social graph is what drives adoption of watch parties, which is a core feature of MatchWatch. Seeing friends' activity (reviews, watchlists) creates a constant reason to return to the app, increasing retention.
- **Chat:** Keeping communication inside the app removes the need to switch to external platforms, increasing session length. It also serves as the coordination layer for watch parties, making it a dependency for that feature's success.

3 Dependencies and Assumptions

After thinking through the dependencies myself, I used Claude to verify and refine them.

4 Scenario Design

For each user story, two scenarios were defined: a **normal flow** (the happy path) and an **exceptional flow**.

Friends — Normal: A user searches for another user by username, sends a friend request, and the recipient accepts. Both users can now view each other's profiles and invite each other to watch parties.

Friends — Exceptional: A user searches for a username that does not exist in the system. The app blocks the action and displays a clear error message, without creating any request.

Chat — Normal: A user opens a conversation with a friend, sends a message, and the friend receives it in real time. The message appears in both users' conversation threads.

Chat — Exceptional: A user attempts to send a message while offline. The message fails to deliver, the app notifies the user of the failure, and the message is queued for automatic retry when connectivity is restored.

5 UI Mockups

To visualise the scenarios quickly, I asked Claude to generate HTML mockups based on my scenario descriptions. This gave me a working starting point that I could iterate on

visually rather than building from scratch.

Once the HTML was generated, I went in and adjusted the layout, colours, and content to better match my vision for the MatchWatch app — dark theme, and some other small components.

This approach significantly reduced the time needed to produce realistic-looking mockups compared to designing them entirely by “hand”.

6 Acceptance Tests

Acceptance tests were written in Gherkin (Given/When/Then) format. I used GenAI assistance to help with the initial structure and wording of the tests, then reviewed and adapted each one to accurately reflect the scenarios I had defined.

7 PDF Deliverable

Up until this point all the work was done using markdown, and in order to create a pleasant looking .pdf I asked Claude AI to create a $\text{\LaTeX}$ file based on my .md and proceeded to make some minor changes so that it looked the way it looks at this moment.

8 Critical Analysis

This deliverable was more analytical than the previous one, less about tooling/set up and more about thinking through a feature and what it depends on, how can it be implemented and tested.

The mockup workflow (AI-generated HTML as a starting point and then manual tweaking) worked well. It was significantly faster than designing from scratch in a tool like Figma which I didn't have any previous experience with.